Конспект лекций по дисциплине ЯЗЫКИ ПРОГРАММИРОВАНИЯ

Кириченко Константин Дмитриевич

Институт математики и информационных технологий ИГУ

2025 год

Лекция 8. Языки программирования

Парадигмы программирования. Структурное программирование

- Парадигмы программирования
- Структурное программирования
- Инструкции циклов в структурном программировании
- Вопросы и задания

Парадигмы программирования

- Парадигма — совокупность идей и методов, основанных на устоявшихся алгебро-логических теориях и определяющих синтаксис, семантику и прагматику языка программирования.
- Каждый язык программирования реализует одну или несколько парадигм программирования.
- Примеры парадигм:
  - автоматное программирование;
  - структурное программирование;
  - объектно-ориентированное программирование;
  - шаблонное программирование;
  - функциональное программирование;
  - логическое программирование.

Дизайн языков программирования

- Каждый язык программирования предназначен для решения определенного круга задач.
- Набор используемых парадигм зависит от назначения языка программирования.
- Синтаксис и семантика языка программирования зависит от используемых в нем парадигм.
- Синтаксис языка программирования должен предоставлять наиболее эффективные средства для реализации заданной парадигмы.
- Пример программы для сложения нескольких чисел в парадигме функционального программирования (Python).

```
print(sum(map(int,input().split())))
```

- Примеры решения задачи A+B на различных языках https://ru.wikipedia.org/wiki/A%2BB.

Синтаксический сахар и синтаксическая соль

- Синтаксическим сахаром называют элементы синтаксиса, упрощающие восприятие программ, но не влияющие на их исполнение.
- Пример синтаксического сахара в C++.

```
for (auto &x:vec) {
    ...
}
```

Циклы по коллекции разворачиваются в такую конструкцию

```
for (auto iter=vec.begin(); iter!=vec.end(); ++iter) {
    auto &x=*iter;
    ...
}
```

- Примеры синтаксического сахара в Python

if x==y==z :	if x==y and y==z:
a = [0]*5	a = [0, 0, 0, 0, 0]

Синтаксический сахар и синтаксическая соль

- Некоторые формы синтаксического сахара являются скорее привычкой, например завершение инструкции специальным символом «;».
- Некоторые формы синтаксического сахара действительно упрощают восприятие, например разделение параметров запятой при вызове функции.
- Некоторые формы синтаксического сахара требует заметного усложнения транслятора и не всегда так понятны, как кажутся.
  - Следующие две строчки эквивалентны или нет?

```
if x == y == z:
if (x == y) == z:
```

  - Это двумерный массив или нет?

```
arr = [[0] * 10] * 10
```

Синтаксическая соль

- Синтаксической солью называют элементы синтаксиса, не требуемые с точки зрения выполнения программы, которые используются для дополнительной проверки на наличие ошибок.
- Пример синтаксической соли в C++.

```
bool less(const int a,const int b) {
   return a<b;
}
```

const не влияет на исполнение, но позволяет избежать ошибок

Pythonic

- Одной из целей языка Python заявлено написание коротких, эффектных, легко читаемых программ.
- Любые правила или традиции, мешающие заявленной цели, без колебаний отменяются. (Although practicality beats purity.)
- В сообществе Python такая парадигма программирования называется Pythonic.

unpythonic	pythonic
a<b and b<c	a<b<c
t=a;a=b;b=t;	a,b=b,a

- Что такое Pythonic?

Написание «сахарных» программ? | Важный элемент концепции языка Python?

Должен быть один способ сделать это

There should be one — and preferably only one — obvious way to do it.

- Некоторые парадигмы дополняют друг друга.
- Большая часть языков программирования реализует несколько парадигм.
- Вместе с тем некоторые парадигмы предоставляют различные синтаксические конструкции, таким образом, существуют принципиально различные способы записи программ.

Парадигмы C++.

- Структурное программирование.
- Объектно-ориентированное программирование.
- Шаблонное программирование.

Многоязыковое и мультипарадигменное программирование

- Многоязыковым программированием называется возможность написания в рамках одной среды программирования программ, использующих различные языки программирования.
- При многоязыковом программирование фрагменты, написанные на различных языках, выделены в отдельные модули программы.
- Примером среды многоязыкового программирования является .NET Framework.
- Мультипарадигменным программированием называется возможность использовать в одном языке программирования средства, относящиеся к различным парадигмам.
- Примером мультипарадигменного языка программирования является Python.

Многоязыковое и мультипарадигменное программирование

- Критики мультипарадигменного программирования считают, что использование принципиально различных средств написания программ в пределах одного модуля недопустимо, поскольку разные парадигмы используют различную логику составления алгоритмов.
- Что выводит эта программа?

```
a = list(range(10))
b = [a.pop() for _ in a]
print(b)
```

При конструировании списка в функциональной парадигме использован метод с побочным действием, которого в рамках функционального программирования быть вообще не должно.

Лекция 8. Языки программирования

Парадигмы программирования. Структурное программирование

- Парадигмы программирования
- Структурное программирования
- Инструкции циклов в структурном программировании
- Вопросы и задания

Императивное программирование

- Инструкция — это синтаксическая единица, описывающая определенный набор действий виртуальной машины языка программирования.
- Инструкции делят на простые и составные. Инструкция называется составной, если она может содержать несколько других инструкций.
- В C++ составная инструкция (compound statement) записывается при помощи фигурных скобок и образует блок, используемый, в частности, для определения области действия переменных.
- Языки программирования, построенные на основе инструкций, называют императивными.
- Языки программирования, построенные на основе определений, называют декларативными.
- Наиболее распространенной парадигмой императивного программирования является структурное программирование.

Понятие инварианта

- Инвариант в программировании — это некоторый логический предикат (логическое выражение), связанный с определенной точкой или множеством точек в программе.
- Инвариант должен быть истинным всякий раз, когда программа находится в связанной с ним точке (множеством точек).
- Обычно инвариант — это неекторое логическое выражение, описывающее состояние и взаимосвязь переменных и других объеков в памяти.
- Понятие инварианта используется для обоснования корректности программ, для контроля семантической правильности, для оптимизации программ.

Если инвариант в начале фрагмента программы $(a = x) \wedge (b = y)$

```
if (a>b)
   swap(a, b)
```

то инвариант после $(\{a, b\} = \{x, y\}) \wedge (a \leq b)$

Структурное программирование

- Структурное программирование воспринимается как описание пошагового процесса преобразования исходных данных к выходным данным.
- Программа записывается как последовательность инструкций, среди которых могут быть составные инструкции.
- Инвариант, который имеет место перед выполнением некоторой инструкции, называется предусловием.
- Аналогично, инвариант, который имеет место после выполнения некоторой инструкции, называют постусловием.
- Если две инструкции выполняются последовательно, то постусловие первой инструкции является предусловием второй.

Пример инвариантов в простой программе

Программа	Пред/постусловие
int a,b,c;	$(cin = [x, y])$
cin >> a >> b;	$(a = x) \wedge (b = y)$
c=a+b;	$(a = x) \wedge (b = y) \wedge (c = x + y)$
a=c-a;	$(a = y) \wedge (b = y) \wedge (c = x + y)$
b=c-a;	$(a = y) \wedge (b = x) \wedge (c = x + y)$
cout << a << b;	$(cout = [y, x])$

Составные инструкции в структурном программировании

- В рамках структурного программирования все инструкции либо размещаются последовательно, либо вкладываются друг в друга с использованием составной инструкции.
- Для составной инструкции предусловие должно выполняться в момент входа в блок, а постусловие — в момент выхода из блока.
- Для вложенных инструкций также есть свои пред/постусловия.
- Таким образом, парадигма структурного программирования предполагает метод пошаговой детализации (программирование сверху вниз), при котором задача разбивается на линейную последовательность подзадач, каждая из которых, в свою очередь, может быть разбита на более мелкие подзадачи.
- Если постусловие блока достигнуто в его середине, то можно выполнить выход из него при помощи инструкции break (если блок является циклом) или return (если блок является функцией).
- В Java инструкция break позволяет выполнить выход из любого именованного блока.

Инструкция ветвления

- Инструкция ветвления используется, если требуемое постусловие достигается различными способами для разных вариантов входных данных.
- В Python используется инструкция множественного ветвления.

```
if ... :
elif ... :
else :
```

- В C++ ветвление только бинарное.

```
if (...)
else
```

- Но множественное ветвление можно реализовать при помощи бинарного.

```
if (...)
else if (...)
else
```

Лекция 8. Языки программирования

Парадигмы программирования. Структурное программирование

- Парадигмы программирования
- Структурное программирования
- Инструкции циклов в структурном программировании
- Вопросы и задания

Цикл по коллекции

- Цикл по коллекции в C++ имеет следующий вид.

```
for (type x : collection)
```

- Если требуется изменять содержимое коллекции, то можно использовать ссылки.

```
for (type &x : collection)
```

- Цикл можно применить к любой коллекции элементов, возвращающей итераторы начала и конца последовательности при помощи функций `begin()` и `end()`.
- В Python аналогичный цикл имеет следующий вид.

```
for x in collection:
```

- Цикл по коллекции также называют одновременным циклом.

Цикл со счетчиком

- Постусловие в цикле со счетчиком всегда достигается после фиксированного, известного заранее числа повторений.
- Счетчик в цикле, как правило, хранит номер текущей итерации и не должен изменяться иным способом, кроме стандартного увеличения.
- В C++ цикл со счетчиком записывается при помощи инструкции for, возможности которой значительно шире. Дисциплина применения цикла должна обеспечиваться программистом.
- В Python цикл со счетчиком реализован при помощи цикла по последовательности и объекта `Range`, который позволяет получить элементы некоторой арифметической прогрессии.
- Цикл по коллекции или цикл со счетчиком может восприниматься как линейная последовательность повторяющихся вычислений, свернутая для удобства в цикл.

Инструкции цикла с условием

- Цикл должен выполняться, пока не будет достигнуто его постусловие.
- Проверка постусловия (или его части) может записываться как логическое выражение.
- Цикл с предварительной проверкой (цикл с предусловием) имеет следующий вид.

```
while ( Постусловие цикла не достигнуто )
  ...
```

- Цикл с проверкой после итерации (цикл с постусловием) имеет следующий вид.

```
do
  ...
while ( Постусловие цикла не достигнуто );
```

Использование инструкции break в циклах

- Постусловие в цикле может достигаться несколькими способами. В этом случае можно использовать инструкцию `break` по следующей схеме.

```
while ( Постусловие цикла не достигнуто ) {
    if ( Постусловие цикла достигнуто )
        break;
}
```

- Использование break для такой цели не нарушает принципы структурного программирования, но в случае больших циклов может ухудшить читаемость программ.

Использование else в циклах на Python

- В Python существует возможность дописать фрагмент кода в том случае, если выход из цикла произошел по предусловию или в связи с концом последовательности. Для этого используется ключевое слово `else`.
- Такая возможность удобна, если для каждого способа достижения постусловия требуется некоторый код завершения блока.

```
s=[1,2,3,4,5]
for i in range(n):
    if s[i]==0:
        print('yes',i)
        break
else:
    print('no')
```

Цикл Дейкстры и цикл Паук

- Цикл Дейкстры и цикл Паук — конструкции циклов, которая могут быть реализованы при помощи `break`.

do
$P_1 \to S_1$
...
$P_n \to S_n$
od

```
for (;;) {
    if (P1) {S1;}
    else if (P2) {S2;}
    else break;
}
```

- Далее записан упрощенный цикл Паук.

do
$P_1 \to S_1$
$P_n \to S_n$
out
$Q_1 \to T_1$
$Q_n \to T_m$
od

```
for (;;) {
    if (P1) {S1;}
    else if (P2) {S2;}
    else if (Q1) {T1;break;}
    else if (Q2) {T2;break;}
    else break;
}
```

Инвариант цикла

- Инвариантом цикла называется инвариант, который имеет место при входе в цикл и в момент завершения каждой итерации цикла.
- Постусловие цикла получается как конъюнкция инварианта и предиката, задающего условие выхода из цикла.
- Условие выхода из цикла является отрицанием условия продолжения цикла.

```
n = int(input())
k = 1
p = 0
while k<n:
    k*=2
    p+=1
```

- Инвариант цикла $k = 2^p$
- Предикат выхода из цикла $k \geq n$
- Постусловие цикла $(k = 2^p) \wedge (k \geq n)$
- Следствие из постусловия $2^p \geq n$. То есть, программа находит степень двойки, большую или равную, чем n.

Числа Фибоначчи

- Последовательность чисел Фибоначчи определяется рекуррентной формулой

$$F(0) = 0; F(1) = 1; F(n) = F(n-1) + F(n-2)$$

Дано число n, надо найти n-е число Фибоначчи, $n \geq 1$

- Будем искать числа циклом, за каждую итерацию находить одно новое число.
- В переменной `i` будем хранить номер числа. В переменной `x` — i-е число Фибоначчи.
- Нам потребуется еще $i-1$-е число. Для него возьмем переменную `y`.
- Инвариант цикла: $(x = F(i)) \wedge (y = F(i-1))$
- Условие выхода из цикла: $i = n$.
- Постусловие цикла: $(x = F(i)) \wedge (y = F(i-1)) \wedge (i == n)$, из которого следует, что $x = F(n)$, то есть x — ответ.

Числа Фибоначчи

- Чтобы инвариант выполнялся при входе в цикл зададим $i = 1$, $x = F(i) = F(1) = 1$, $y = F(i-1) = F(0) = 0$
- Как записать тело цикла?
- Пусть в начале итерации цикла $i = k$, тогда инвариант примет вид $(i = k) \wedge x = F(k) \wedge y = F(k-1)$.
- Цикл завершится увеличением `i` на 1, то есть будет выполняться $i = k + 1$.
- Тогда, чтобы инвариант выполнялся нам потребуется найти такой набор преобразований, чтобы после его выполнения имело место $(x = F(k+1)) \wedge (y = F(k))$.
- Попробуем взять присваивание `x+=y`, получим $x = F(k) + F(k-1) = F(k+1)$, то есть $(x = F(k+1)) \wedge (y = F(k-1))$. Заметим, что $F(k) = F(k+1) - F(k-1)$, то есть теперь можно взять присваивание `y=x-y`

Числа Фибоначчи

```
n = int(input())
x = 1
y = 0
for i in range(1,n):
    x+=y
    y=x-y
print(x)
```

Объявления переменных

- Переменные в C++ должны объявляться в том блоке, где они используются. Это связано с тем, что переменные удаляются при выходе из блока, что позволет экономить память и избегать ошибок, связанных с обращениями к переменным вне их блока.
- Как правило, переменные следует явно инициализировать в момент объявления. Наиболее распространенным исключением из этого правила в C++ является ввод значений из потока, расположенный непосредственно после объявления переменных.

```
int a,b,c;
cin >> a >> b >> c;
```

- Цикл for в C++ создает блок. Поэтому объявление переменных в этом цикле является правильной практикой.

```
for (int i=0,j=n; i<j; i++,j--)
    ...
```

Лекция 8. Языки программирования

Парадигмы программирования. Структурное программирование

- Парадигмы программирования
- Структурное программирования
- Инструкции циклов в структурном программировании
- Вопросы и задания

- Практикум по Python (часть 4).

Напишите программы на языке Python. Задания размещены в тестирующей системе ejudge.
`http://olymp3.isu.ru/cgi-bin/new-client?contest_id=103`
Чтобы программы были засчитаны, их необходимо защитить во время практики. Тексты всех программ следует сохранить в файле отчета. Вопросы будут касаться синтаксиса языка и используемых алгоритмов решения.

1. Рассмотрим программу на Python

```
x = [int(t) for t in input().split()]
for i in range(len(x)):
    if x[i]<i or x[i]>2*i:
        break
print(i)
```

Перепишите эту программу так, чтобы в ней использовался цикл `while`, в ней не использовалась инструкция `break`, а используемый алгоритм не изменился. Полученная программа должна иметь следующий вид

```
x = [int(t) for t in input().split()]
i = 0
while ...:
    i+=1
print(i)
```

Допишите правильное логическое выражение вместо многоточия. **Выражение не должно содержать оператор** `not`.

2. (2) Рассмотрим программу на Python

```
x = [int(t) for t in input().split()]
for i in range(len(x)):
    if i<=x[i]<=2*i or 3*i<=x[i]<=4*i:
        break
print(i)
```

Перепишите эту программу так, чтобы в ней использовался цикл `while`, в ней не использовалась инструкция `break`, а используемый алгоритм не изменился. Полученная программа должна иметь следующий вид

```
x = [int(t) for t in input().split()]
i = 0
while ...:
    i+=1
print(i)
```

Допишите правильное логическое выражение вместо многоточия. **Выражение не должно содержать оператор** `not`.

3. Напишите пример программы на Java с двумя вложенными циклами и инструкцией break для выхода из двух вложенных циклов. В качестве такой программы можно взять поиск первого нуля в двумерном массиве.
4. Запишите программу из лекции и проверьте ее работу.

```
a = list(range(10))
b = [a.pop() for _ in a]
print(b)
```

Объясните, почему список b получился именно таким.

5. Напишите программу на C++, которая прочитает натуральное число n, создаст вектор из n элементов, прочитает значения элементов, добавит последний элемент вектора к каждому элементу (в том числе и к последнему), найдет и выведет количество положительных и отрицательных элементов вектора после изменения. Из циклов в программе можно использовать только циклы по коллекции.

6. Представим, что в некотором будущем стандарте C++ появится конструкция

```
forr(i, from, to)
```

которая будет раскрываться в следующий код

```
{
    int __i = from;
    for(const int &i=__i, __n=to; __i < __n; __i++) {
        ...
    }
}
```

Какой вид цикла реализует эта конструкция? Почему?

7. Напишите программу на языке Python для проверки того, что все числа во введенной последовательности неотрицательные. Если в цикле будет найдено отрицательное число, цикл должен прерваться и вывести найденное число. Если отрицательных чисел нет, то вывести некоторое сообщение об этом **Программа должна использовать конструкцию for break else**.

8. **(2)** Рассмотрите следующую программу. Опишите множества значений при которых программа выводит 0, 1 и 2. Например, $A0 = (x, y)|x \in [5; \infty), y \in [-5; \infty.$

```
if x>=5 and y>=-5:
    print(0):
elif x<0 or y<0:
    print(1):
else:
    print(2):
```

9. **(2)** Напишите программу на языке Python для решения следующей задачи. Задана пронумерованная последовательность логических значений $t(0), t(1), \cdots, t(n)$, $t(i) \in \{False, True\}$ Последовательность обладает следующим свойством: существует некоторое число k такое, что все значения $t(0), t(1), \cdots, t(k-1)$ равны *False*, а все значения $t(k), \cdots t(n)$ равны *True*, то есть слева от k находятся значения *False*, а справа — *True*. Последовательность задается функцией `t`, которая возвращает логическое значение. Например, для тестирования эту функцию можно задать следующим образом

```
def t(i)
    return i>=1000
```

Требуется написать программу, которая найдет число k. Программа должна реализовывать алгоритм бинарного поиска. Диапазон поиска задается переменными left и right. На каждом шаге вычисляется центр диапазона, значение в нем проверяется, после чего диапазон сокращается в два раза.

Алгоритм должен быть записан через цикл `while` и реализовывать инвариант $(t(left) = False) \wedge (T(right) = True)$.

```
left = 0
right = 10**18
while ...:
    mid = ...
    if t(mid):
        ...
    else:
        ...
print(...)
```